

Labeling Training Data for Entity Matching Using LLMs

Aaron Steiner

Data and Web Science Group

University of Mannheim

Mannheim, Germany

aaron.steiner@uni-mannheim.de

Christian Bizer

Data and Web Science Group

University of Mannheim

Mannheim, Germany

christian.bizer@uni-mannheim.de

Abstract

Supervised entity matching systems depend on labeled record pairs, but manually creating such training data remains a costly step in data integration projects. Recent large language models can label individual entity pairs in zero-shot settings, raising a practical question: can the manual labeling step for training-data construction be automated? This paper studies an LLM-based training-set construction workflow that selects candidate pairs from two source tables, labels the selected pairs with an LLM, and uses the resulting data to train downstream entity matching models. We evaluate the workflow by replacing the official training splits of standard EM benchmarks with selected-and-labeled training sets while keeping the downstream matcher, test split, and evaluation code fixed. Preliminary results show that active-learning-based selection with LLM labels matches or outperforms the official training split on several product benchmarks and remains close on most bibliographic benchmarks. The results also show that selection matters: the best active-learning strategy is consistently stronger than simpler similarity-based selection and can reduce labeling cost by spending the LLM budget on positives and boundary-relevant negatives. We analyze the generated training sets at the pair, entity, label, and difficulty levels and find that the strongest sets are not merely noisy reconstructions of the benchmark splits; they change the training distribution in ways that explain downstream gains. These findings position LLM-based selection and labeling as a practical method for constructing EM training data, while also showing that performance claims need to be paired with an audit of what the constructed training sets contain.

Keywords

entity matching, training-set construction, large language models, data labeling, data quality

1 Introduction

Entity matching (EM) is a core step in data integration projects: given two collections of records, decide which pairs refer to the same real-world entity [4, 5]. Modern EM systems increasingly rely on supervised models, including transformer-based matchers [1, 2, 9, 10], but these models need labeled pairs before they can be deployed in a new domain [8, 11]. Creating such training data is expensive because useful labels must include not only obvious matches and non-matches, but also boundary cases: same-family products, compatible components, near-duplicate publication titles, and other pairs that are easy to confuse [3, 11].

Large language models (LLMs) make it possible to revisit this bottleneck. Prior work has shown that LLMs can directly classify entity pairs in zero-shot settings and can be fine-tuned for EM [12,

14]. In this paper, we study a different, more operational question. If a strong LLM can label pairs, is it still necessary to manually label training data for every new EM task, or can the training-data construction step be automated?

We answer this question by studying whether an LLM-based system can *build an entire training dataset by selecting pairs from the two source tables and labeling those pairs*. The goal is not to replace the downstream matcher with an LLM at inference time. Instead, the LLM is used as a data construction tool: it labels selected examples once, after which the resulting training set can be used to train a conventional downstream matcher such as Ditto or a smaller local model.

This distinction matters because a useful training-set construction system has to solve two coupled problems. First, it must select which candidate pairs should be included in the final training set. A random or nearest-neighbor-only strategy may spend most of the annotation budget on easy negatives, while missing positives or hard non-matches. Second, it must label the selected pairs. The LLM component is responsible for assigning match/non-match labels, but the downstream value of the selected-and-labeled training set depends just as much on the selection component as on the labeling component.

The main question of the paper is therefore practical:

Can an LLM-based training-set construction system replace manual EM training-data labeling by selecting and labeling training sets that match or beat the official training sets of standard benchmarks?

We answer this question empirically by constructing selected-and-labeled training sets for multiple EM benchmarks, training downstream matchers on these selected-and-labeled sets, and comparing their performance to matchers trained on the official benchmark training splits. Preliminary results show that selected-and-labeled training sets can perform equally or better than official training sets on several benchmarks. The second part of the paper then asks why this happens. We compare selected-and-labeled and official training sets by pair overlap, entity coverage, class balance, similarity and difficulty distributions, and qualitative categories of selected examples.

The central finding is that the result is not only a labeling result. Selection matters. A simple similarity-search strategy can spend much of its budget on repetitive or easy negatives. An active-learning strategy, especially one that uses the downstream matcher in the loop, selects more positives and more decision-boundary examples. This changes both downstream F1 and labeling cost. We therefore treat the workflow as a combined *selection-and-labeling* problem rather than as a pure LLM classification problem.

The paper is organized around five research questions.

- RQ1** Can selected-and-labeled training sets match or outperform official benchmark training sets for downstream EM?
- RQ2** Which example-selection strategies and training-set sizes are most effective?
- RQ3** How expensive is LLM-based training-set construction, and how does cost differ across selection strategies?
- RQ4** Do the findings depend on one proprietary LLM labeler, or do they transfer to other strong LLMs, including open-weight models?
- RQ5** How do selected-and-labeled training sets differ from official training sets at the pair, entity, label, and difficulty levels, and which differences explain downstream performance gains or losses?

The intended contributions are:

- an LLM-based system for selecting and labeling EM training data that combines candidate selection with LLM labeling;
- an empirical comparison of selected-and-labeled versus official training sets across standard EM benchmarks;
- evidence that active-learning-based example selection produces more useful training data than simpler similarity-based selection;
- a cost analysis showing that targeted selection can reduce the amount of LLM labeling needed;
- a robustness analysis plan across different LLM labelers, including an open-weight labeling model for settings where data cannot be sent to a hosted API;
- a detailed analysis methodology for explaining what selected-and-labeled training sets contain and how they differ from official splits.

2 Background and Problem Setting

2.1 Entity Matching

Entity matching is the task of deciding whether two descriptions from different data sources refer to the same real-world entity. It is a central step in data integration pipelines, because downstream tasks such as price comparison, product catalog construction, knowledge graph construction, and bibliographic data integration depend on consistent entity identifiers [4, 5]. In the pairwise formulation used throughout this paper, each candidate pair (l, r) consists of one record from a left table and one record from a right table. A label $y \in \{0, 1\}$ indicates whether the two records match.

Most practical EM pipelines separate candidate construction from pair classification. Candidate construction, often called blocking, reduces the Cartesian product of two tables to a smaller set of plausible pairs. Pair classification then uses rules, similarity features, supervised classifiers, or neural models to decide whether each candidate pair is a match. Classical approaches include probabilistic record linkage [7], rule-based duplicate detection, and supervised feature-based systems [3, 6]. Workflow-oriented systems such as Magellan highlight that EM is not only a classifier, but an end-to-end engineering process involving data preparation, blocking, labeling, debugging, and evaluation [8].

Neural EM methods replaced hand-crafted similarity features with learned representations. Early deep matching systems explored distributed tuple representations and neural pair classifiers [10].

Later systems adopted transformer encoders and pretrained language models (PLMs), such as BERT or RoBERTa, to serialize records into text and classify record pairs [2, 9]. Ditto is a representative PLM-based EM system that combines pretrained encoders with EM-specific data augmentation and domain knowledge injection [9]. These systems reach strong benchmark performance, but they still require task-specific labeled pairs and can be brittle when test pairs contain entities that were not observed during training [12].

2.2 LLM-Assisted Auto-Labeling

Large language models (LLMs) provide a different interface to EM. Instead of fine-tuning a compact encoder on thousands of labeled pairs, a model can be prompted with two serialized entity descriptions and asked whether they refer to the same entity. Prior work by Peeters, Steiner, and Bizer showed that LLMs can perform competitively in zero-shot and few-shot EM settings, but also that there is no universally best prompt: prompt design and model choice interact strongly with the dataset [12]. The same work showed that LLMs can produce structured explanations for matching decisions and use these explanations to support error analysis.

Follow-up work on fine-tuning LLMs for EM showed that fine-tuning improves smaller models and in-domain transfer, while cross-domain generalization remains mixed [14]. That study also explored training-example representation, including textual and structured explanations, and LLM-assisted example filtering and selection. These results motivate a closer look at the selected-and-labeled training data itself: if LLMs are used to label selected examples, the resulting labels should be treated as data artifacts with measurable provenance and quality.

The auto-labeling setting considered in this paper starts from a source benchmark and constructs a new labeled training set from pairs drawn from the two source tables. A pipeline first builds candidate pairs using retrieval or similarity search, then selects the most useful pairs for labeling, and finally asks an LLM to classify each selected pair. The output is a labeled dataset that can be used to train or evaluate downstream matchers.

This setting differs from direct LLM inference on a fixed test set. Direct inference asks whether an LLM can solve EM cases one at a time. Auto-labeling asks whether selected and LLM-labeled pairs form a training artifact with enough quality, coverage, and provenance for reliable downstream use. The latter requires auditing the selected-and-labeled data itself.

2.3 Benchmark Setting

The experiments in this draft follow the benchmark families used in the prior LLM-for-EM studies: product matching datasets such as WDC Products, Abt-Buy, Walmart-Amazon, and Amazon-Google, and bibliographic datasets such as DBLP-ACM and DBLP-Scholar [12, 14]. These datasets expose different failure modes. Product matching often contains sparse or noisy descriptions, variants of the same product, and hard non-matches with similar surface forms. Bibliographic matching contains author, title, venue, and year variants, where small textual differences can be either harmless formatting variation or evidence for distinct publications.

Source records → candidate pool → example selection → LLM labeling
 → selected-and-labeled training set → downstream EM training and
 training-set analysis.

Figure 1: Training-set construction system. The central design choice is the combination of example selection and LLM labeling.

2.4 Data Quality Risks

LLM-assigned labels introduce several risks that are not visible from aggregate downstream F1 alone.

- **Split overlap:** selected pairs may duplicate official train, validation, or test pairs, which complicates claims about novelty and can invalidate training/evaluation separation.
- **Coverage bias:** retrieval-based candidate selection can over-sample near-neighbor regions and under-sample hard negatives or rare positive patterns.
- **Class-prior shift:** selected-and-labeled sets may have positive rates that differ from both official training data and target deployment data.
- **Structured noise:** LLM decisions can fail systematically for products with sparse descriptions, bibliographic variants, or cases where attribute agreement is misleading.

The goal of this paper is not only to maximize a downstream metric, but to make these risks measurable.

3 Training-Set Construction System

Figure 1 summarizes the system studied in this paper. The system receives two source tables and constructs a training set for a downstream EM model. It has two main components: an example-selection component and an LLM labeling component. The selection component decides which pairs are worth labeling; the labeling component assigns match/non-match decisions to those pairs.

3.1 Candidate Pool Construction

The system first constructs a pool of possible record pairs. For benchmarks where the full Cartesian product is too large, the pool is built using retrieval and similarity search. The current implementation uses embedding-based nearest-neighbor search to obtain likely positives and informative negatives, and it can add random pairs to preserve broader coverage. The resulting pool is larger than the final training set, so the next step must decide which examples should consume the labeling budget.

3.2 Example Selection

The selection component is the part that turns auto-labeling from simple pair classification into training-set construction. It chooses the final examples that will be labeled and exported as a training set. The repository currently supports profiles with different budgets, such as small, medium, large, and all-size profiles, as well as variants that add random examples.

We compare three selection styles. A similarity-search strategy selects nearest-neighbor candidates and therefore tests how far simple retrieval plus LLM labeling can go. A generic active-learning strategy iteratively selects examples using model uncertainty and

disagreement signals. A Ditto-based active-learning strategy uses the downstream matcher inside the selection loop and therefore targets the decision boundary of the model that will later be trained on the generated data. This lets us separate two effects: whether the LLM can label pairs accurately, and whether the system selects a useful training distribution.

3.3 LLM Labeling

Each selected pair is serialized into a prompt that presents the relevant record attributes and asks for a binary match decision. The system stores the final label together with run metadata such as prompt usage, model configuration, batch identifiers, and parse status. The main experiments use one strong LLM labeler to search for the best construction method. This keeps the comparison between selection strategies controlled: if two selected-and-labeled training sets differ in downstream performance while using the same labeler, the difference is likely caused by the selected examples rather than by a better labeling function.

After identifying the strongest construction method, we use additional LLM labelers as a robustness check. This includes an open-weight model for the labeling step. The purpose is not to run an exhaustive model leaderboard, but to test whether the workflow depends on a single hosted API model. In practical deployments, a large model can be used for offline labeling, while the trained downstream matcher can remain a cheaper BERT- or Ditto-style model for production inference.

3.4 Training-Set Materialization

After labeling, the system materializes the selected-and-labeled dataset in formats usable by downstream matchers, including Ditto-compatible training files. The selected-and-labeled set is treated as a replacement candidate for the official training split: the downstream model is trained either on the official training data or on the selected-and-labeled training data, and both are evaluated on the same official test split. This replacement design is the cleanest test of the practical question: can automatically constructed training data remove or reduce the need for manual labels?

3.5 Analysis Layer

The system also records enough provenance to inspect the selected-and-labeled training data itself. We compare selected-and-labeled and official training sets by exact pair overlap, entity coverage, positive rate, similarity distributions, and difficulty classes. This analysis is not secondary bookkeeping; it is necessary for explaining why a selected-and-labeled set can match or beat the official set.

4 Comparing Selected-and-Labeled and Official Training Sets

The first empirical claim of the paper is about downstream performance: selected-and-labeled training sets can match or slightly outperform official training sets. The second claim is explanatory: selected-and-labeled sets differ from official splits in systematic ways. This section defines the training-set comparison used to support that explanation.

4.1 Pair-Level Overlap

For each selected pair, we compute a canonical pair key from the left and right record identifiers. Exact-key joins against official train, validation, and test splits measure how much of the selected-and-labeled data reuses existing benchmark pairs. Pair-level overlap answers two different questions. Overlap with the official training split indicates how much of the selected-and-labeled set is a reconstruction of known training examples. Overlap with validation or test splits flags cases that must be handled carefully when using selected-and-labeled data for training.

4.2 Entity Coverage and Class Balance

Pair overlap alone does not describe the training distribution. A selected-and-labeled set can reuse many entities while still creating different pair combinations. We therefore measure coverage over left and right entities, the number of pairs per entity, and the positive rate. This is important because selected-and-labeled training data should contain enough positive examples to teach the model what a match looks like, while also including hard negatives that prevent over-generalization.

4.3 Similarity and Difficulty

The central hypothesis is that example selection changes downstream performance by changing the difficulty mix. We therefore analyze textual and embedding similarity between records in a pair. For negative examples, high similarity often indicates a hard non-match; for positive examples, low similarity often indicates a hard match. We also classify selected examples into qualitative difficulty categories such as easy negative, hard same-family negative, accessory-vs-main-product negative, straightforward positive, and boundary positive.

4.4 Consistency and Label Quality

When cluster identifiers or other weak correctness signals are available, we use them as proxy checks for label quality. These checks are not treated as ground truth; they are used to identify likely false positives, likely false negatives, and examples that merit manual inspection. The final paper should report label-quality checks separately from training-set usefulness, because a selected-and-labeled set can be useful for downstream training while still containing systematic noise that should be understood.

5 Experimental Design

5.1 Benchmarks

The experimental plan uses standard EM benchmark families represented in the repository. The product benchmarks are ABT-BUY, AMAZON-GOOGLE, and WALMART-AMAZON; the bibliographic benchmarks are DBLP-ACM and DBLP-SCHOLAR. These domains have different attribute structures and error profiles.

5.2 Training Sets

For each benchmark, we compare at least two training sources.

- **Official:** the benchmark-provided training split.
- **Selected-and-labeled:** pairs selected from the two source tables and labeled with the LLM.

For benchmarks with multiple selected-and-labeled profiles, we compare different training budgets and selection strategies. The main comparison separates the base selection strategies from later quality-improvement variants. The base strategies are similarity search, generic active learning, and Ditto-based active learning. The refinement table then starts from the strongest base strategy and tests relabeling and consistency-based dropping. This separation keeps the empirical story readable: first identify which selection family works best, then ask whether label cleaning improves it further.

5.3 Downstream Matchers

The primary evaluation trains the same downstream EM architecture on different training sets and evaluates all models on the same official test split. Ditto is the natural first downstream matcher because the repository already contains Ditto export and training support. The downstream model is fixed when comparing official and selected-and-labeled training sets; otherwise, performance differences could be caused by model changes rather than training-data quality.

We additionally plan a deployment-oriented comparison with a small local LLM matcher. This is not the main paper claim, but it answers a practical question: if an organization has more inference compute than a BERT-style deployment requires, does a compact local LLM recover additional F1 after the training labels have been constructed? These experiments should be reported separately from the main replacement experiment.

5.4 Evaluation Metrics

The main model metric is test-set F1, with precision and recall reported to explain tradeoffs. We also report accuracy where useful, but F1 is more informative for imbalanced EM datasets. For selected-and-labeled training data, we additionally report selection and dataset metrics: training-set size, positive rate, exact overlap with official splits, entity coverage, similarity distributions, difficulty classes, and proxy label-quality indicators.

For the construction process, we report LLM labeling cost in addition to model performance. Cost is measured from recorded token usage and model pricing metadata where available. The cost table reports API labeling cost only; local retrieval, embedding computation, and active-learning training overhead are discussed separately because they depend on the local infrastructure.

5.5 Main Comparison

The main result table has one row per benchmark and training source:

- benchmark and test split;
- training source: official, selected-and-labeled, and optionally official plus selected-and-labeled;
- training-set size and positive rate;
- downstream precision, recall, and F1;
- F1 difference between selected-and-labeled and official training.

This table supports the paper’s first claim directly: selected-and-labeled training sets can match or slightly outperform the official

Table 1: Preliminary pair-level comparison between selected-and-labeled training sets and official benchmark splits. The table shows that selected-and-labeled sets vary substantially in how much they reproduce official training pairs and how much genuinely new pair coverage they add.

Benchmark	Profile	Rows	Train overlap (%)	Novel pairs (%)	Valid overlap (%)	Test overlap (%)
abt-buy	small	1,000	71.00	11.70	16.80	17.30
abt-buy	medium	3,000	71.43	11.20	17.40	17.37
abt-buy	large	5,000	70.54	12.16	17.60	17.30
amazon-google	all	7,680	56.52	31.02	14.23	12.46
dblp-acm	all	9,442	42.99	47.80	10.69	9.21
dblp-scholar	all	22,809	33.93	60.83	8.48	5.24
walmart-amazon	all	7,547	66.00	24.66	16.40	9.34

training sets. The later analysis sections then explain what is inside those selected-and-labeled sets.

5.6 Robustness Across LLM Labelers

The main method search uses one strong LLM labeler so that the selection-strategy comparison is not confounded by model changes. After selecting the strongest workflow, we relabel the resulting selected pairs with additional models and, where feasible, rerun the full active-learning workflow with an open-weight LLM. The planned comparison has two levels. First, we measure direct label agreement on the selected training pairs to estimate how many labels would change under another model. Second, we train the downstream matcher on the relabeled data to test whether these label differences are large enough to change F1.

The key robustness claim is modest: the workflow should work with a sufficiently capable LLM labeler and should not require one particular proprietary model. If the open-weight model reaches comparable performance, the paper can also support a deployment argument for settings where source data cannot be sent to an external API.

6 Preliminary Results and Analyses

This section establishes the main empirical story in four steps. First, selected-and-labeled training sets are compared against official benchmark training sets. Second, we show that the selection strategy matters. Third, we analyze labeling cost and planned model robustness checks. Fourth, we explain the performance differences by inspecting the composition of the generated training data.

6.1 RQ1: Selected-and-Labeled Versus Official Training Sets

The first result compares downstream matchers trained on official training data with the same matcher trained on selected-and-labeled training data. This is the central claim of the paper. We separate the result into two parts. Table 2 compares the main ways of selecting pairs before LLM labeling. Table 3 then focuses only on refinements of the best-performing selection family.

The important methodological constraint is that the downstream model, test split, and evaluation code must remain fixed. Only the training set changes. This makes the comparison a direct test of

whether the selected-and-labeled training set can replace the official training split.

We use Active learning (ML) for the generic active-learning selector and Active learning (Ditto) for the variant that uses Ditto during the selection loop.

Table 2 shows the main selection result. Among the selected-and-labeled training sets, Active learning (Ditto) is the best strategy on all five benchmarks. It improves over the official training set on ABT-BUY and WALMART-AMAZON, is close to the official training set on DBLP-ACM and WDC, and remains below the official training set on DBLP-SCHOLAR. The result supports the practical claim that manual benchmark labels can often be replaced by automatically selected and LLM-labeled pairs, but it also shows that the replacement is not automatic. The pair-selection policy determines whether the labeling budget produces a useful training distribution.

This result also clarifies the scope of the claim. The strongest claim is not that any LLM-labeled set works. The strongest claim is that a targeted active-learning workflow can produce competitive training data with no manual pair labels for the new task.

Table 3 shows that improving the label quality of the Active learning (Ditto) set can yield additional gains, but the best refinement depends on the benchmark. Relabeling gives the strongest result on ABT-BUY and WDC. Closure-based dropping is strongest on WALMART-AMAZON. The bibliographic benchmarks show less benefit from these refinements, suggesting that their official training sets may already cover the relevant decision boundary more effectively than the selected-and-labeled variants. We therefore treat these refinements as optional quality-control steps rather than as separate construction methods.

6.2 RQ2: Effect of Example Selection

The second result compares selected-and-labeled sets that use the same LLM labeler but different selection strategies. This separates the value of the selection component from the value of the LLM component. If two selected-and-labeled sets are labeled by the same LLM but produce different downstream performance, the explanation should lie in which examples were selected.

For ABT-BUY, the current repository already supports this analysis by comparing a seed-only similarity expansion against the richer Active learning (Ditto) strategy. This comparison is especially useful because both strategies target the same benchmark and can be inspected at the pair level.

Table 2: Main selection strategies. Each cell reports F1 $\pm$ standard deviation over three runs. The non-official columns train Ditto on pairs selected from the two source tables and labeled with the LLM-based system.

Benchmark	Official set	Similarity search	Active learning (ML)	Active learning (Ditto)
ABT-BUY	88.30 $\pm$ 1.50	87.60 $\pm$ 0.60	87.70 $\pm$ 0.90	90.10 $\pm$ 0.20
DBLP-ACM	98.40 $\pm$ 0.30	98.10 $\pm$ 0.20	97.50 $\pm$ 0.40	98.10 $\pm$ 0.30
DBLP-SCHOLAR	95.60 $\pm$ 0.30	93.70 $\pm$ 0.40	94.00 $\pm$ 0.30	94.10 $\pm$ 0.10
WALMART-AMAZON	85.70 $\pm$ 1.00	85.50 $\pm$ 2.10	86.70 $\pm$ 0.10	86.90 $\pm$ 0.40
WDC	71.90 $\pm$ 1.00	69.50 $\pm$ 0.40	71.10 $\pm$ 0.50	71.70 $\pm$ 0.90

Table 3: Quality-improvement variants for Active learning (Ditto). Each cell reports F1 $\pm$ standard deviation over three runs. The variants start from the Active learning (Ditto) selected-and-labeled set and then either relabel uncertain examples, drop examples implicated by closure checks, or combine both filters.

Benchmark	Official	AL-Ditto	+Relabel	+Closure drop	+Closure or relabel drop	+Closure and relabel drop	+Relabel drop
ABT-BUY	88.30 $\pm$ 1.50	90.10 $\pm$ 0.20	92.10 $\pm$ 0.40	89.60 $\pm$ 0.90	92.10 $\pm$ 0.60	89.60 $\pm$ 0.90	90.60 $\pm$ 0.50
DBLP-ACM	98.40 $\pm$ 0.30	98.10 $\pm$ 0.30	97.70 $\pm$ 0.20	98.20 $\pm$ 0.40	98.20 $\pm$ 0.40	98.10 $\pm$ 0.10	97.90 $\pm$ 0.20
DBLP-SCHOLAR	95.60 $\pm$ 0.30	94.10 $\pm$ 0.10	93.70 $\pm$ 0.80	92.90 $\pm$ 0.60	93.40 $\pm$ 0.60	94.00 $\pm$ 0.30	93.90 $\pm$ 0.70
WALMART-AMAZON	85.70 $\pm$ 1.00	86.90 $\pm$ 0.40	87.20 $\pm$ 0.60	88.50 $\pm$ 0.90	88.00 $\pm$ 1.20	87.40 $\pm$ 0.50	88.50 $\pm$ 0.20
WDC	71.90 $\pm$ 1.00	71.70 $\pm$ 0.90	73.10 $\pm$ 0.90	69.70 $\pm$ 2.50	70.40 $\pm$ 0.70	71.30 $\pm$ 1.20	72.20 $\pm$ 0.50

6.3 RQ3: Cost of Training-Set Construction

Table 4 compares the LLM labeling cost of the three selected-and-labeled construction strategies. The table uses the token and cost metadata stored with the runs. Where the run stored token usage but not the final dollar cost, we compute the cost using the same gpt-5.2 pricing assumptions recorded in the other manifests. The table reports API labeling cost only; it does not include local retrieval, embedding computation, or the additional Ditto training cost used inside Active learning (Ditto).

The cost results reinforce the main selection result. Active learning (Ditto) is not only the strongest selection strategy in Table 2; for the benchmarks with complete cost metadata, it also uses less LLM labeling budget than the similarity-search baseline. The reason is that similarity search is much less targeted. It retrieves many plausible near neighbors, but many of these pairs are easy or repetitive negatives. Finding additional positives therefore requires labeling many more pairs overall, which raises the total cost while still failing to recover as many positive examples as the active-learning strategy. Active learning (Ditto), in contrast, uses model disagreement and decision-boundary signals to spend the LLM budget on pairs that are more likely to improve the final training set, including positives and hard negatives. The missing WDC cost should be recovered from the original run logs before the final submission.

6.4 RQ4: Robustness Across LLM Labelers

The current method search uses one strong LLM labeler for the main results. This is deliberate: the selection-strategy comparison in Table 2 should not mix changes in example selection with changes in the labeling function. A compact robustness section then tests whether the workflow depends on this one model.

The planned experiment has two parts. First, we run the strongest selected-pair sets through additional labelers and report label agreement with the main LLM labels. This includes at least one open-weight model. Second, we train the downstream matcher on one or two relabeled sets and compare F1 against the main LLM-labeled

result. If the label flips are rare and downstream F1 remains close, the paper can claim that the workflow transfers across sufficiently capable labelers. If the open-weight model performs similarly, the practical implication is stronger: proprietary source data can be labeled with a local model and then used to train a cheaper downstream matcher.

This robustness experiment is scoped narrowly. The paper does not need a large model leaderboard. The purpose is to show that the selection-and-labeling workflow is not an artifact of one hosted model.

6.5 RQ5: Training-Set Composition

To understand why two selected-and-labeled training sets can lead to different downstream behavior, we inspect their contents rather than only their final F1 scores. We use ABT-BUY as a detailed case study because it is a product-matching benchmark with many plausible near misses: same-brand model variants, accessories, kits, compatible components, and category-level substitutes. We compare the training data selected by Active learning (Ditto) with a simpler seed-only expansion based on similarity search. Both data sets are labeled with the same LLM setup, so differences in downstream performance should not be attributed to a better labeling function. Instead, they should be explained by which examples each strategy selects.

For the main comparison we use the all_plus20random profile. The seed-only profile contains 6,000 unique pairs, while the Active learning (Ditto) profile contains 5,759 consistent unique pairs after excluding two ambiguous duplicate keys. The two sets share 2,604 pairs. Thus, 3,396 pairs are exclusive to the seed-only set and 3,155 consistent pairs are exclusive to the Active learning (Ditto) set. The positive class already shows a strong asymmetry: 795 seed positives are also positive in the Active learning (Ditto) set, corresponding to 97.55% of the seed positives, but the Active learning (Ditto) set contains many additional positives not present in the seed-only set.

We then classify the exclusive examples by their *informational value*. First, we use an LLM as a qualitative classifier. The classifier

Table 4: LLM labeling cost for constructing selected-and-labeled training sets. Cost is the recorded or reconstructed API cost in USD. Cost per 1k pairs normalizes by the number of selected pairs in the profile used for the downstream result.

Benchmark	Method	Selected pairs	LLM cost (USD)	Cost / 1k pairs
ABT-BUY	Similarity search	6,000	3.69	0.62
ABT-BUY	Active learning (ML)	5,000	2.36	0.47
ABT-BUY	Active learning (Ditto)	4,039	1.85	0.46
DBLP-ACM	Similarity search	10,800	7.40	0.69
DBLP-ACM	Active learning (ML)	9,442	3.98	0.42
DBLP-ACM	Active learning (Ditto)	5,000	3.00	0.60
DBLP-SCHOLAR	Similarity search	24,000	13.48	0.56
DBLP-SCHOLAR	Active learning (ML)	22,809	9.22	0.40
DBLP-SCHOLAR	Active learning (Ditto)	12,000	6.02	0.50
WALMART-AMAZON	Similarity search	4,369	5.53	1.27
WALMART-AMAZON	Active learning (ML)	7,547	3.33	0.44
WALMART-AMAZON	Active learning (Ditto)	3,600	1.72	0.48
WDC	Similarity search	10,000	15.57	1.56
WDC	Active learning (ML)	20,000	10.83	0.54
WDC	Active learning (Ditto)	14,521	–	–

receives the existing pair label and is instructed not to relabel the pair; instead, it assigns the pair to a difficulty or relation class such as easy negative, hard same-family negative, accessory-vs-main-product negative, same-ecosystem component negative, straight-forward positive, or boundary positive. Second, to validate that this LLM-based characterization is not merely an artifact of the classifier prompt, we run a traditional token-overlap baseline. This baseline computes an extended Jaccard score over title, description, and price fields. For negative examples, higher overlap indicates greater difficulty; for positive examples, lower overlap indicates greater difficulty. Thresholds are tuned separately for positives and negatives using tertiles.

Table 5 shows that the difference is mainly a selection effect. The similarity-search expansion mostly adds negative coverage: 3,395 of its 3,396 exclusive examples are negative, and 27.47% of its exclusive examples are easy negatives according to the LLM classifier. In contrast, Active learning (Ditto) adds 309 positive examples that are not present in the similarity-search set. This is the strongest distributional difference between the two training sets. The LLM classifier further shows that Active learning (Ditto) examples are slightly more concentrated in hard or domain-specific negative classes, including accessory-vs-main-product, same-ecosystem component, and narrow-category high-similarity cases.

The Jaccard baseline supports the same interpretation. Among negative exclusive examples, the similarity-search set contains a much larger low-difficulty region: 40.38% of similarity-search negatives are low difficulty under the Jaccard metric, compared with only 25.40% for Active learning (Ditto). Conversely, Active learning (Ditto) has more medium- and high-difficulty negatives: 37.53% and 37.07%, compared with 29.90% and 29.72% for similarity search. This traditional similarity-based analysis agrees with the LLM-based result: the active-learning procedure spends less of its budget on easy negatives and more on boundary-relevant examples.

6.6 Explaining Downstream Performance

The ABT-BUY analysis suggests that Active learning (Ditto) does not merely produce a larger or differently labeled training set. It changes the training distribution. It preserves most of the seed positive signal while adding many additional positives and a higher share of boundary-relevant product confusions. The similarity-search strategy, by contrast, provides broad negative coverage but spends a larger fraction of its exclusive budget on easy negatives.

This provides a plausible data-centric explanation for why a selected-and-labeled training set can match or outperform an official training set: the selected-and-labeled set may contain a more useful mix of positives and hard negatives for the downstream model. The next analysis step is to connect this explanation directly to the main F1 table by checking whether benchmarks with better selected-training performance also exhibit better difficulty balance, positive coverage, or entity coverage.

7 Discussion

7.1 Why Selected-and-Labeled Training Data Can Work

The core argument is not simply that LLMs can label entity pairs. The stronger claim is that a system can construct useful EM training data when it combines a capable labeler with a selection strategy that chooses informative examples. The selection component determines whether the training set contains positives, hard negatives, and boundary cases that teach the downstream model useful decision boundaries. The LLM component then makes it possible to label those selected examples without manual annotation at benchmark scale.

This distinction is important for the paper’s positioning. A plain “LLM labels pairs” story would mostly be another LLM-for-EM classification result. The more data-management-oriented contribution is that the system decides which pairs should be labeled in the first place. The ABT-BUY analysis shows why this matters:

Table 5: Composition of exclusive ABT-BUY training examples for similarity search versus Active learning (Ditto). LLM groups aggregate per-example informational-value classes. The extended-Jaccard rows provide a token-overlap validation of the LLM-based analysis.

Signal among exclusive examples	Similarity search	AL-Ditto	Difference
Exclusive examples	3,396	3,157	–
Positive examples	1 (0.03%)	309 (9.79%)	+9.76 pp
Easy negatives (LLM)	933 (27.47%)	361 (11.43%)	-16.04 pp
Hard negatives (LLM)	1,540 (45.35%)	1,535 (48.62%)	+3.27 pp
Moderate/domain negatives (LLM)	922 (27.15%)	950 (30.09%)	+2.94 pp
Positive signal classes (LLM)	1 (0.03%)	309 (9.79%)	+9.76 pp
Low-difficulty negatives (Jaccard)	1,371 / 3,395 (40.38%)	723 / 2,846 (25.40%)	-14.98 pp
Medium-difficulty negatives (Jaccard)	1,015 / 3,395 (29.90%)	1,068 / 2,846 (37.53%)	+7.63 pp
High-difficulty negatives (Jaccard)	1,009 / 3,395 (29.72%)	1,055 / 2,846 (37.07%)	+7.35 pp
High-difficulty positives (Jaccard)	–	101 / 309 (32.69%)	–

active learning preserves the useful positive signal from simpler retrieval while adding many additional positives and a larger share of boundary-relevant negatives.

7.2 Selected-and-Labeled Data Is Not Automatically Better

Selected-and-labeled training data can fail in several ways. The selection strategy may over-sample easy negatives, miss rare positives, or duplicate existing benchmark pairs. The LLM labeler may introduce systematic false positives or false negatives. A selected-and-labeled set can also perform well on a downstream test while containing artifacts that make the result hard to interpret. For this reason, the paper presents downstream performance and training-set composition together.

The refinement results also argue against a single universal cleaning recipe. Relabeling helps some product datasets, closure-based dropping helps others, and the bibliographic datasets appear less sensitive to these refinements. Cleaning is therefore a quality-control layer around the best selection method, not the main contribution.

7.3 Model Choice and Deployment

The main experiments can use one strong hosted LLM labeler to keep the selection comparison controlled. For deployment, however, the model choice has two separate roles. The labeling model can be large and expensive because it is used offline to build a finite training set. The production matcher can remain a cheaper Ditto/BERT-style model, or a compact local LLM if the extra inference cost is justified by higher F1 on a specific dataset.

The planned open-weight labeler experiment supports the privacy and deployment argument. If a sufficiently capable open-weight model produces similar labels and downstream F1, organizations with proprietary data can run the labeling step locally instead of sending source records to an external API. This is a robustness and applicability result, not a claim that every smaller model will work equally well.

7.4 Positioning Against Prior Work

Prior work from this group studied direct LLM matching and fine-tuning LLMs for EM [12, 14]. This paper is positioned as the next step: using LLM systems to select and label training examples for conventional downstream matchers. That framing makes the contribution a data construction and data management contribution, not only another LLM-for-classification study.

7.5 Threats to Validity

The current analysis uses benchmark-specific identifiers, split definitions, and proxy difficulty measures. Exact pair overlap does not capture all forms of entity reuse, and cluster-consistency checks are only proxies for correctness. The remaining validation work includes a manual review sample, especially for pairs classified as hard positives or hard negatives. The analysis also separates three quantities: label correctness, novelty relative to official splits, and downstream usefulness. Finally, the strongest method has so far been selected mainly with one LLM labeler; the robustness section needs to make clear how much the conclusions depend on that labeler.

8 Related Work

Entity matching foundations. Entity matching has a long history in statistics and data integration, from probabilistic record linkage [7] and duplicate detection surveys [6] to book-length treatments of data matching and entity resolution [3, 5]. Recent surveys frame EM as an end-to-end process for large-scale data integration rather than a standalone binary classifier [4]. Magellan follows this view by supporting a full EM workflow with blocking, labeling, debugging, and evaluation [8].

Neural and PLM-based matchers. Deep learning broadened the design space for EM by learning representations of serialized records and record pairs [1, 10]. Transformer-based matchers and PLM-based systems then became strong baselines for benchmark EM tasks [2, 9]. Ditto is especially relevant for this paper because it is a widely used downstream matcher for training-data experiments and supports data augmentation and knowledge injection [9]. The limitation shared by these supervised systems is their dependence

on task-specific labels, which motivates both LLM-based direct matching and LLM-assisted construction of labeled training data.

LLMs for entity matching. Recent work studies whether LLMs can replace or complement PLM-based EM systems. Peeters, Steiner, and Bizer evaluate hosted and open-source LLMs for zero-shot, few-shot, rule-augmented, and fine-tuned EM, showing high zero-shot performance, strong robustness to unseen entities, but also model- and dataset-specific prompt sensitivity [12]. Their work also demonstrates structured explanations and LLM-assisted error analysis. Steiner, Peeters, and Bizer further investigate fine-tuning LLMs for EM, showing that fine-tuning can improve smaller models and in-domain transfer, while cross-domain transfer and example selection effects are mixed [14]. The present paper builds on those findings but changes the unit of analysis: instead of asking only whether LLMs can classify pairs or be fine-tuned for EM, it asks whether selected and LLM-labeled pairs form a reliable, auditable training dataset.

Weak supervision and data programming. Auto-labeling is related to weak supervision, where noisy labeling functions are combined to create training data [13]. In EM, weak supervision can reduce manual labeling effort, but it also makes label provenance and noise modeling central concerns. The difference in this paper is that the labeling source is an LLM-driven EM pipeline and the focus is on auditing selected pair labels against benchmark structure, including split overlap, novelty, coverage, and consistency proxies.

Benchmarking and artifact quality. EDBT-style data management evaluation rewards not only accuracy but also reproducibility, artifacts, and careful benchmark interpretation. This paper follows that tradition by treating LLM-assigned labels as persistent artifacts. The proposed audit connects model evaluation to data quality checks that can be rerun before using LLM-assigned labels in downstream experiments.

9 Conclusion

This paper studies whether LLMs can remove or reduce the manual labeling step in supervised entity matching. The proposed workflow selects candidate pairs from two source tables, labels them with an LLM, and uses the resulting selected-and-labeled data to train a downstream matcher. The preliminary results show that targeted active-learning-based construction can match or outperform official training splits on several benchmarks, while simpler similarity-based selection is less reliable.

The main lesson is that automated EM training-data construction is a selection-and-labeling problem. A capable LLM labeler is necessary, but not sufficient; the system also needs to spend the labeling budget on positives and boundary-relevant negatives. The training-set audit explains this effect by showing that the strongest selected-and-labeled sets differ systematically from simpler retrieval-based sets and from official splits. The remaining experiments should complete the cost table, add a compact robustness check with additional LLM labelers including an open-weight model, and connect the training-set composition analysis more directly to the benchmark-level F1 results.

10 Artifacts

The code, selected-pair label summaries, audit scripts, and downstream evaluation outputs are maintained in the accompanying repository. The final submission should provide a stable artifact snapshot with instructions for reproducing the tables in this paper. At minimum, the artifact should include the exact selected-and-labeled pair files, official split hashes or source references, audit scripts, model-training configuration, and result aggregation scripts.

TODO before submission: create an archive or release tag, remove credentials and machine-local paths, document hardware and model versions, and provide a single command or notebook for recreating the paper tables.

References

- [1] Nils Barlaug and Jon Atle Gulla. 2021. Neural Networks for Entity Matching: A Survey. *ACM Transactions on Knowledge Discovery from Data* 15, 3 (2021), 52:1–52:37.
- [2] Ursin Brunner and Kurt Stockinger. 2020. Entity Matching with Transformer Architectures: A Step Forward in Data Integration. In *Proceedings of the 23rd International Conference on Extending Database Technology*. OpenProceedings.org, Konstanz, Germany, 463–473.
- [3] Peter Christen. 2012. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, Berlin, Heidelberg.
- [4] Vassilis Christophides, Vasilis Efthymiou, Themis Palpanas, George Papadakis, and Kostas Stefanidis. 2020. An Overview of End-to-End Entity Resolution for Big Data. *Comput. Surveys* 53, 6 (2020), 127:1–127:42.
- [5] Vassilis Christophides, Vasilis Efthymiou, and Kostas Stefanidis. 2015. *Entity Resolution in the Web of Data*. Morgan & Claypool, San Rafael, CA, USA.
- [6] Ahmed K. Elmagarmid, Panagiotis G. Ipeirotis, and Vassilios S. Verykios. 2007. Duplicate Record Detection: A Survey. *IEEE Transactions on Knowledge and Data Engineering* 19, 1 (2007), 1–16.
- [7] Ivan P. Fellegi and Alan B. Sunter. 1969. A Theory for Record Linkage. *J. Amer. Statist. Assoc.* 64, 328 (1969), 1183–1210.
- [8] Pradap Konda, Sanjib Das, Paul Suganthan, AnHai Doan, Ardian Ardalan, Jeffrey R. Ballard, Han Li, Fatemah Panahi, Haojun Zhang, Jeffrey F. Naughton, Shishir Prasad, Ganesh Krishnan, Rohit Deep, and Vijay Raghavendra. 2016. Magellan: Toward Building Entity Matching Management Systems. *Proceedings of the VLDB Endowment* 9, 13 (2016), 1581–1584.
- [9] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. 2020. Deep Entity Matching with Pre-Trained Language Models. *Proceedings of the VLDB Endowment* 14, 1 (2020), 50–60.
- [10] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. 2018. Deep Learning for Entity Matching: A Design Space Exploration. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 19–34.
- [11] Ralph Peeters, Reng Chiz Der, and Christian Bizer. 2024. WDC Products: A Multi-Dimensional Entity Matching Benchmark. In *Proceedings of the 27th International Conference on Extending Database Technology (EDBT 2024)*. OpenProceedings.org, Konstanz, Germany, 22–33. doi:10.48786/edbt.2024.03
- [12] Ralph Peeters, Aaron Steiner, and Christian Bizer. 2025. Entity Matching using Large Language Models. In *Proceedings of the 28th International Conference on Extending Database Technology (EDBT 2025)*. OpenProceedings.org, Konstanz, Germany, 529–541. doi:10.48786/edbt.2025.42
- [13] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid Training Data Creation with Weak Supervision. *Proceedings of the VLDB Endowment* 11, 3 (2017), 269–282.
- [14] Aaron Steiner, Ralph Peeters, and Christian Bizer. 2025. Fine-Tuning Large Language Models for Entity Matching. In *2025 IEEE 41st International Conference on Data Engineering Workshops (ICDEW 2025)*. IEEE Computer Society, Los Alamitos, CA, USA, 9–17. doi:10.1109/ICDEW67478.2025.00006